

ROADMAP BELAJAR FULL-STACK 2026

Go + Vue.js + PostgreSQL + Git + Docker

Dari fundamental yang samar-samar sampai mampu menyelesaikan latihan algoritma dan membangun aplikasi full-stack sederhana.

Lapisan	Teknologi	Peran
Editor	VS Code	Menulis, menjalankan, dan debug kode
Backend	Go (Golang)	Logika aplikasi, API, akses database
Frontend	Vue.js + Node.js/npm	Antarmuka web dan tooling frontend
Database	PostgreSQL + DBeaver	Penyimpanan data dan administrasi database
API Testing	Postman / Insomnia	Menguji endpoint HTTP
Version Control	Git + GitHub	Riwayat perubahan dan kolaborasi
Container	Docker Desktop	Menjalankan service secara terisolasi
Runtime	Browser modern	Menjalankan dan debug aplikasi Vue

■ Target latihan inti

Setelah bagian fundamental, kamu harus mampu: (1) mengecek bilangan prima, (2) menghitung rata-rata elemen dalam array/slice, dan (3) mencari nilai terbesar dalam array/slice - tanpa menyalin solusi.

Dokumen ini disusun sebagai buku belajar berurutan. Jangan berusaha menghafal semua syntax. Fokus pada alasan di balik setiap langkah, lalu ketik ulang contoh dan kerjakan latihan tanpa melihat jawaban.

Cara Menggunakan Buku Ini

Gunakan urutan bab. Setiap konsep baru dibangun dari konsep sebelumnya. Untuk setiap contoh kode, lakukan tiga tahap: baca, ketik ulang, lalu modifikasi sendiri.

Simbol belajar	Arti
Konsep	Hal yang perlu dipahami sebelum menulis kode.
Kode	Contoh yang perlu diketik ulang, bukan sekadar dibaca.
Cek diri	Pertanyaan singkat untuk memastikan pemahaman.
Latihan	Soal untuk dikerjakan tanpa melihat solusi.
Checkpoint	Kemampuan minimum sebelum lanjut ke tahap berikutnya.

Aturan latihan

1. Ketik kode secara manual minimal satu kali. Copy-paste hanya setelah kamu mengerti bentuknya.
2. Sebelum coding, tulis input, output, dan langkah algoritma dalam bahasa biasa.
3. Jika error, baca pesan error dari baris pertama yang relevan sebelum mengubah kode secara acak.
4. Uji program dengan data normal, data batas, dan data yang berpotensi membuat program gagal.
5. Simpan latihan dengan Git sehingga kamu dapat melihat perkembanganmu dari waktu ke waktu.

Daftar Isi

Bagian	Materi
1	Peta Belajar dan Cara Kerja Program
2	Algoritma, Pseudocode, dan Problem Solving
3	Setup Go dan Program Pertama
4	Variable, Tipe Data, Konstanta, dan Operator
5	Input/Output dan Konversi Tipe
6	Boolean dan Percabangan
7	Perulangan dengan for
8	Array, Slice, String, dan Map
9	Function, Scope, dan Multiple Return
10	Struct, Pointer, Method, dan Error Dasar
11	Latihan Inti: Prima, Rata-rata, Nilai Terbesar
12	Debugging, Testing, dan Cara Membaca Error
13	Git dan GitHub dari Dasar
14	Dasar Web: HTTP, JSON, REST, dan API
15	Backend Go: HTTP Server dan API Sederhana
16	JavaScript Fundamental untuk Vue
17	Vue 3 Fundamental
18	SQL dan PostgreSQL
19	DBeaver, Postman, dan Browser DevTools
20	Docker Fundamental
21	Menggabungkan Vue + Go + PostgreSQL
22	Mini Project Full-Stack

Bagian	Materi
23	Jadwal Belajar dan Checkpoint
24	Bank Latihan + Kunci Pembahasan
A	Cheat Sheet Go
B	Cheat Sheet Git, SQL, HTTP, dan Docker

Peta Belajar dan Cara Kerja Program

Mulai dari model mental sederhana: program menerima input, memprosesnya, lalu menghasilkan output.

1.1 Model input - process - output

```

CODE
INPUT
↓
PROCESS
↓
OUTPUT

```

Contoh: pengguna memasukkan 10 dan 20. Program menjumlahkannya. Outputnya 30. Model ini berlaku dari latihan sangat kecil sampai aplikasi web besar.

1.2 Dari latihan kecil ke full-stack

```

CODE
Browser / Vue
  ↓ HTTP + JSON
Go REST API
  ↓ SQL
PostgreSQL

```

Di aplikasi full-stack, input dapat berasal dari formulir Vue. Go memvalidasi dan memprosesnya. PostgreSQL menyimpan data. Output kembali sebagai JSON dan ditampilkan oleh Vue.

1.3 Urutan belajar yang disarankan

Tahap	Fokus	Selesai jika...
A	Logika + Go dasar	Bisa membuat program console dengan if, for, slice, function.
B	Problem solving	Bisa menyelesaikan prima, rata-rata, maksimum/minimum.
C	Go web + HTTP	Bisa membuat endpoint GET/POST sederhana.
D	JavaScript + Vue	Bisa membuat UI dan memanggil API.
E	SQL + PostgreSQL	Bisa CRUD dengan SQL dan relasi dasar.

Tahap	Fokus	Selesai jika...
F	Docker + integrasi	Bisa menjalankan service dan project full-stack.

■ Prinsip utama

Jangan belajar Vue, Docker, dan database secara bersamaan ketika loop dan function masih membingungkan. Fondasi algoritma akan dipakai di semua teknologi tersebut.

Algoritma, Pseudocode, dan Problem Solving

Koding adalah terjemahan dari langkah penyelesaian masalah. Jika langkahnya belum jelas, syntax tidak akan menyelamatkan program.

2.1 Memecah soal

Untuk setiap soal, jawab pertanyaan berikut sebelum menulis kode:

1. Apa inputnya?
2. Apa output yang diminta?
3. Data apa yang perlu disimpan?
4. Apakah perlu percabangan?
5. Apakah perlu perulangan?
6. Apakah ada rumus atau aturan?
7. Apa edge case-nya?
8. Bagaimana menguji hasilnya?

2.2 Contoh: mencari angka terbesar

Data: [5, 9, 2, 15, 7]. Kita tidak meminta komputer "langsung tahu" 15. Kita memberi langkah yang bisa diulang.

```
PSEUDOCODE
terbesar = elemen pertama
untuk setiap nilai:
    jika nilai > terbesar:
        terbesar = nilai
tampilkan terbesar
```

2.3 Trace table

Nilai yang dicek	Terbesar sebelum	Apakah lebih besar?	Terbesar sesudah
5	5	Tidak perlu	5
9	5	Ya	9
2	9	Tidak	9
15	9	Ya	15
7	15	Tidak	15

■ Teknik penting

Saat bingung dengan loop, buat trace table. Catat nilai variabel setiap iterasi. Ini memaksa logika program menjadi terlihat.

2.4 Kompleksitas dasar

Untuk tahap awal cukup pahami bahwa loop yang membaca n elemen sekali memiliki pekerjaan yang tumbuh sebanding dengan n , sering ditulis $O(n)$. Dua loop bersarang sering menuju $O(n^2)$. Kamu belum perlu menghafal teori kompleksitas secara mendalam, tetapi biasakan memperhatikan berapa kali data dibaca.

Setup Go dan Program Pertama

Tujuan bab ini adalah memastikan lingkungan bekerja dan kamu mengerti struktur minimal program Go.

3.1 Verifikasi instalasi

```
• • • TERMINAL
go version
code --version
git --version
```

Jika perintah go tidak dikenali, perbaiki instalasi/PATH sebelum lanjut. Belajar akan jauh lebih sulit jika environment belum stabil.

3.2 Membuat project

```
• • • TERMINAL
mkdir belajar-go
cd belajar-go
go mod init belajar-go
```

File go.mod menandai root module. Untuk latihan kecil, satu module dapat berisi banyak file atau folder latihan.

3.3 Program pertama

```
• • • GO
package main

import "fmt"

func main() {
    fmt.Println("Halo Go")
}
```

package main menyatakan program executable. import "fmt" membawa fungsi format/input-output. func main() adalah titik awal program ketika dijalankan.

```
• • • TERMINAL
go run .
```

3.4 gofmt

```
• • • TERMINAL
gofmt -w .
```

Go memiliki format standar. Biasakan memakai gofmt sehingga indentasi dan spacing konsisten.

3.5 Struktur folder latihan

```
CODE
belajar-go/
├── go.mod
├── 01-variable/
│   └── main.go
├── 02-if/
│   └── main.go
└── 03-loop/
    └── main.go
```

Variable, Tipe Data, Konstanta, dan Operator

Semua algoritma membutuhkan tempat menyimpan keadaan. Variable adalah nama untuk data yang dapat berubah.

4.1 Deklarasi variable

```
GO
var umur int = 17
nama := "Andi"
tinggi := 170.5
aktif := true
```

Tipe	Contoh	Kegunaan
int	17	Bilangan bulat
float64	170.5	Bilangan desimal
string	"Andi"	Teks
bool	true / false	Kondisi logika

4.2 Zero value

Variable yang dideklarasikan tanpa nilai awal memiliki zero value.

```
GO
var jumlah int // 0
var harga float64 // 0
var nama string // ""
var aktif bool // false
```

4.3 Konstanta

```
GO
const Pi = 3.14159
const AppName = "Belajar Go"
```

Gunakan const untuk nilai yang tidak seharusnya berubah selama program berjalan.

4.4 Operator aritmatika

Operator	Makna	Contoh
+	Tambah	$10 + 3 = 13$
-	Kurang	$10 - 3 = 7$
*	Kali	$10 * 3 = 30$
/	Bagi integer jika operand int	$10 / 3 = 3$
%	Sisa bagi	$10 \% 3 = 1$

■ Penting untuk soal prima

Operator % memberi sisa pembagian. Jika $n \% \text{pembagi} == 0$, berarti n habis dibagi oleh pembagi tersebut.

4.5 Operator perbandingan dan logika

```
GO
a > b
a < b
a >= b
a <= b
a == b
a != b

aktif && terverifikasi
admin || operator
!aktif
```

Hasil operasi perbandingan adalah bool. Operator && berarti AND, || berarti OR, dan ! membalik nilai boolean.

Input/Output dan Konversi Tipe

Program console menjadi interaktif ketika dapat membaca input, lalu mengubah data ke tipe yang sesuai.

5.1 Output dengan fmt

```
GO
fmt.Println("Halo")
fmt.Printf("Umur: %d\n", umur)
fmt.Printf("Rata-rata: %.2f\n", rataRata)
```

Format	Untuk
%d	integer
%f	float
%.2f	float 2 angka desimal
%s	string
%t	boolean
%v	nilai umum

5.2 Input sederhana

```
GO
package main

import "fmt"

func main() {
    var angka int

    fmt.Print("Masukkan angka: ")
    fmt.Scan(&angka)

    fmt.Println("Kamu memasukkan:", angka)
}
```

Tanda & akan lebih masuk akal setelah belajar pointer. Untuk saat ini, pahami bahwa `fmt.Scan` membutuhkan lokasi variable yang akan diisi.

5.3 Konversi tipe

```
GO
total := 5
jumlah := 2

rataRata := float64(total) / float64(jumlah)
fmt.Println(rataRata) // 2.5
```

■ Kesalahan umum

Jika total dan jumlah bertipe int, total / jumlah melakukan pembagian integer. 5 / 2 menghasilkan 2, bukan 2.5.

Boolean dan Percabangan

Percabangan memilih jalur program berdasarkan kondisi true atau false.

6.1 if, else if, else

```
GO
nilai := 85

if nilai >= 90 {
    fmt.Println("A")
} else if nilai >= 80 {
    fmt.Println("B")
} else {
    fmt.Println("C")
}
```

6.2 Ganjil/genap

```
GO
angka := 14

if angka%2 == 0 {
    fmt.Println("Genap")
} else {
    fmt.Println("Ganjil")
}
```

6.3 Kondisi gabungan

```
GO
umur := 19
punyakartu := true

if umur >= 17 && punyaKartu {
    fmt.Println("Syarat terpenuhi")
}
```

6.4 switch

```
• • • GO
hari := 2

switch hari {
case 1:
    fmt.Println("Senin")
case 2:
    fmt.Println("Selasa")
default:
    fmt.Println("Tidak valid")
}
```

6.5 Latihan

- Cek apakah sebuah angka positif, negatif, atau nol.
- Buat program kelulusan dengan batas nilai 75.
- Cek apakah tahun habis dibagi 4. Lalu cari aturan lengkap tahun kabisat sebagai tantangan.
- Buat klasifikasi suhu: dingin, normal, panas dengan batas yang kamu tentukan lalu jelaskan batasnya.

Perulangan dengan for

Loop mengulang pekerjaan. Ketiga latihan utama membutuhkan loop untuk memeriksa pembagi atau membaca elemen array/slice.

7.1 for dengan counter

```
GO
for i := 1; i <= 5; i++ {
    fmt.Println(i)
}
```

Baca: mulai i = 1; selama i <= 5; setelah satu iterasi, i bertambah satu.

7.2 Accumulator

```
GO
total := 0

for i := 1; i <= 5; i++ {
    total += i
}

fmt.Println(total) // 15
```

Accumulator adalah variable yang mengumpulkan hasil dari iterasi ke iterasi. Pola ini akan digunakan untuk total dan rata-rata.

7.3 Loop seperti while

```
GO
i := 1
for i <= 5 {
    fmt.Println(i)
    i++
}
```

7.4 break dan continue

```
GO
for i := 1; i <= 10; i++ {
    if i == 7 {
        break
    }
    if i%2 == 0 {
        continue
    }
    fmt.Println(i)
}
```

break menghentikan loop. continue melewati sisa kode pada iterasi saat ini dan lanjut ke iterasi berikutnya.

7.5 Loop bersarang

```
GO
for baris := 1; baris <= 3; baris++ {
    for kolom := 1; kolom <= 3; kolom++ {
        fmt.Println(baris, kolom)
    }
}
```

■ Cek diri

Jika kamu belum bisa memprediksi nilai i dan total pada setiap iterasi tanpa menjalankan program, buat trace table sampai pola loop terasa jelas.

Array, Slice, String, dan Map

Slice adalah struktur yang paling sering dipakai untuk kumpulan data sederhana di Go.

8.1 Array

```
GO
angka := [5]int{10, 20, 30, 40, 50}
fmt.Println(angka[0]) // 10
```

Panjang array adalah bagian dari tipenya. [5]int dan [6]int adalah tipe berbeda.

8.2 Slice

```
GO
angka := []int{10, 20, 30}
angka = append(angka, 40)

fmt.Println(angka)
fmt.Println(len(angka))
```

8.3 range

```
GO
numbers := []int{10, 20, 30}

for index, value := range numbers {
    fmt.Println(index, value)
}

for _, value := range numbers {
    fmt.Println(value)
}
```

Gunakan `_` ketika suatu nilai sengaja tidak dipakai. Compiler Go akan menolak variable lokal yang dideklarasikan tetapi tidak digunakan.

8.4 Slicing

```
GO
numbers := []int{10, 20, 30, 40, 50}

fmt.Println(numbers[1:4]) // 20 30 40
fmt.Println(numbers[:3])  // 10 20 30
fmt.Println(numbers[2:])  // 30 40 50
```

8.5 String

```
GO
nama := "Go!ang"
fmt.Println(len(nama))
fmt.Println(nama[0])
```

Index string secara langsung bekerja pada byte. Untuk teks Unicode yang kompleks, pelajari rune. Untuk fundamental awal, pahami perbedaan ini agar tidak menganggap len selalu sama dengan jumlah karakter manusia.

8.6 Map

```
GO
nilai := map[string]int{
    "Andi": 90,
    "Budi": 82,
}

fmt.Println(nilai["Andi"])

skor, ada := nilai["Citra"]
fmt.Println(skor, ada)
```

Map menyimpan pasangan key-value. Bentuk nilai, ada := map[key] adalah pola penting untuk membedakan key yang tidak ditemukan.

8.7 Latihan slice

- Jumlahkan semua elemen.
- Hitung berapa elemen yang genap.
- Cari nilai terkecil.
- Balik urutan slice tanpa fungsi helper.
- Cari apakah nilai target ada.
- Buat slice baru berisi hanya nilai > 10.

Function, Scope, dan Multiple Return

Function membuat program lebih mudah dibaca, diuji, dan digunakan kembali.

9.1 Function sederhana

```
GO
func tambah(a int, b int) int {
    return a + b
}

func main() {
    hasil := tambah(10, 20)
    fmt.Println(hasil)
}
```

9.2 Parameter tipe sama

```
GO
func tambah(a, b int) int {
    return a + b
}
```

9.3 Multiple return

```
GO
func bagi(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("pembagi tidak boleh nol")
    }
    return a / b, nil
}
```

Go sering menggunakan pasangan hasil + error. Ini lebih eksplisit daripada menyembunyikan kegagalan.

9.4 Scope

```
GO
func main() {
    x := 10

    if x > 5 {
        pesan := "besar"
        fmt.Println(pesan)
    }

    // pesan tidak tersedia di sini
}
```

9.5 Decomposition

Daripada membuat main sangat panjang, pecah tanggung jawab: `readInput()`, `validateInput()`, `calculateAverage()`, `printResult()`. Kemampuan memecah masalah menjadi function adalah jembatan menuju backend yang rapi.

Struct, Pointer, Method, dan Error Dasar

Bab ini memberi fondasi Go yang diperlukan sebelum membuat API dan berinteraksi dengan database.

10.1 Struct

```
GO
type User struct {
    ID    int
    Name  string
    Age   int
}

func main() {
    user := User{ID: 1, Name: "Andi", Age: 20}
    fmt.Println(user.Name)
}
```

10.2 Pointer

```
GO
func tambahSatu(n *int) {
    *n = *n + 1
}

func main() {
    angka := 10
    tambahSatu(&angka)
    fmt.Println(angka) // 11
}
```

&angka mengambil alamat variable. *n mengakses nilai pada alamat itu. Pointer memungkinkan function mengubah nilai yang dimiliki pemanggil dan menghindari copy besar pada beberapa kasus.

10.3 Method

```
GO
type User struct {
    Name string
}

func (u User) Greeting() string {
    return "Halo, " + u.Name
}
```

10.4 Error handling

```
• • • GO
result, err := bagi(10, 0)
if err != nil {
    fmt.Println("Terjadi error:", err)
    return
}
fmt.Println(result)
```

■ Pola Go

Biasakan memeriksa err segera setelah operasi yang dapat gagal. Jangan membiarkan error diabaikan hanya agar kode terlihat singkat.

Latihan Inti: Prima, Rata-rata, Nilai Terbesar

Ini adalah target latihan dari tugasmu. Pelajari algoritma, implementasi, edge case, dan cara mengujinya.

11.1 Cek bilangan prima - konsep

Bilangan prima adalah bilangan bulat lebih besar dari 1 yang hanya memiliki pembagi positif 1 dan dirinya sendiri. Jika ditemukan pembagi lain, angka tersebut bukan prima.

```

• • • PSEUDOCODE
input: n

jika n < 2:
    bukan prima

untuk i dari 2 sampai sebelum n:
    jika n habis dibagi i:
        bukan prima

prima

```

Versi dasar

```

• • • GO
func isPrime(n int) bool {
    if n < 2 {
        return false
    }

    for i := 2; i < n; i++ {
        if n%i == 0 {
            return false
        }
    }

    return true
}

```

Mengapa return false boleh langsung dilakukan? Karena menemukan satu pembagi selain 1 dan dirinya sudah cukup membuktikan bahwa n bukan prima.

Versi lebih efisien

```
GO
func isPrime(n int) bool {
    if n < 2 {
        return false
    }

    for i := 2; i*i <= n; i++ {
        if n%i == 0 {
            return false
        }
    }

    return true
}
```

Jika n memiliki faktor lebih besar dari akar n , pasangannya harus lebih kecil dari akar n . Karena itu cukup mengecek sampai $i*i \leq n$.

Input	Hasil yang diharapkan	Alasan singkat
-5	false	Bilangan < 2
0	false	Bukan prima
1	false	Bukan prima
2	true	Prima terkecil
3	true	Tidak punya pembagi lain
4	false	Habis dibagi 2
17	true	Tidak ditemukan pembagi
49	false	7 x 7

11.2 Menghitung rata-rata slice

```
GO
numbers := []int{10, 20, 30, 40}

total := 0
for _, number := range numbers {
    total += number
}

average := float64(total) / float64(len(numbers))
fmt.Println(average)
```

Ada dua langkah: akumulasi total, lalu bagi dengan banyak elemen. Konversi ke float64 dilakukan agar pembagian dapat menghasilkan angka desimal.

Versi function yang menangani slice kosong

```
func average(numbers []int) (float64, bool) {
    if len(numbers) == 0 {
        return 0, false
    }

    total := 0
    for _, number := range numbers {
        total += number
    }

    return float64(total) / float64(len(numbers)), true
}
```

bool menandakan apakah hasil valid. Pola ini sederhana untuk latihan; pada aplikasi nyata kamu juga bisa menggunakan error jika kegagalan perlu membawa pesan.

11.3 Mencari nilai terbesar

```
func max(numbers []int) (int, bool) {
    if len(numbers) == 0 {
        return 0, false
    }

    largest := numbers[0]

    for _, number := range numbers[1:] {
        if number > largest {
            largest = number
        }
    }

    return largest, true
}
```

Inisialisasi largest dari elemen pertama. Jangan selalu memulai dari 0, karena data bisa seluruhnya negatif.

■ Contoh bug

Untuk data [-10, -3, -20], jika largest := 0 maka hasil salah: 0 tidak ada di data. Dengan largest := numbers[0], algoritma tetap benar.

11.4 Program gabungan

```
GO
package main

import "fmt"

func isPrime(n int) bool {
    if n < 2 {
        return false
    }
    for i := 2; i*i <= n; i++ {
        if n%i == 0 {
            return false
        }
    }
    return true
}

func average(numbers []int) (float64, bool) {
    if len(numbers) == 0 {
        return 0, false
    }
    total := 0
    for _, number := range numbers {
        total += number
    }
    return float64(total) / float64(len(numbers)), true
}

func max(numbers []int) (int, bool) {
    if len(numbers) == 0 {
        return 0, false
    }
    largest := numbers[0]
    for _, number := range numbers[1:] {
        if number > largest {
            largest = number
        }
    }
    return largest, true
}

func main() {
    n := 17
    fmt.Println("Prima:", isPrime(n))

    numbers := []int{12, 8, 25, 10, 15}

    avg, ok := average(numbers)
    if ok {
        fmt.Println("Rata-rata:", avg)
    }

    largest, ok := max(numbers)
    if ok {
        fmt.Println("Terbesar:", largest)
    }
}
```

11.5 Tantangan tanpa solusi langsung

- Buat isPrime menerima input dari keyboard.
- Buat function min untuk mencari nilai terkecil.
- Hitung jumlah bilangan prima di dalam slice.

- Hitung rata-rata hanya dari angka genap.
- Cari nilai terbesar kedua tanpa sorting.
- Buat function stats yang mengembalikan min, max, dan average.

Debugging, Testing, dan Cara Membaca Error

Programmer tidak hanya menulis kode; sebagian besar waktu digunakan untuk menemukan dan memperbaiki asumsi yang salah.

12.1 Tiga jenis masalah

Jenis	Contoh	Strategi
Compile error	Syntax salah, variable tidak dipakai	Baca file:baris dan pesan compiler.
Runtime error	Index out of range	Cari input yang memicu panic dan validasi batas.
Logic error	Program jalan tetapi hasil salah	Trace variable dan bandingkan dengan hasil yang seharusnya.

12.2 Debug dengan output sementara

```
GO
for i := 2; i*i <= n; i++ {
    fmt.Println("mencoba pembagi:", i)
    if n%i == 0 {
        return false
    }
}
```

Print debugging berguna untuk pemula, tetapi hapus output sementara setelah masalah dipahami.

12.3 Table-driven test

```
• • • GO
package main

import "testing"

func TestIsPrime(t *testing.T) {
    tests := []struct {
        input int
        want  bool
    }{
        {-1, false},
        {0, false},
        {1, false},
        {2, true},
        {4, false},
        {17, true},
        {49, false},
    }

    for _, tc := range tests {
        got := isPrime(tc.input)
        if got != tc.want {
            t.Fatalf("isPrime(%d) = %v, want %v", tc.input, got, tc.want)
        }
    }
}
```

```
• • • TERMINAL
go test ./...
```

12.4 Kebiasaan debugging

1. Reproduksi error dengan input sekecil mungkin.
2. Baca pesan error persis, jangan menebak.
3. Periksa nilai variable sebelum titik gagal.
4. Ubah satu hal pada satu waktu.
5. Tambahkan test agar bug yang sama tidak kembali.

Git dan GitHub dari Dasar

Git mencatat riwayat perubahan lokal. GitHub menjadi remote repository dan tempat kolaborasi.

13.1 Workflow dasar

```
• • • TERMINAL
git init
git status
git add .
git commit -m "feat: latihan dasar Go"
```

Working tree berisi perubahan lokal. Staging area berisi perubahan yang akan masuk commit. Commit adalah snapshot yang diberi pesan.

13.2 Melihat perubahan

```
• • • TERMINAL
git status
git diff
git diff --staged
git log --oneline
```

13.3 Branch

```
• • • TERMINAL
git switch -c latihan-slice
# ubah kode, add, commit
git switch main
git merge latihan-slice
```

13.4 Remote GitHub

```
• • • TERMINAL
git remote add origin <URL-REPOSITORY>
git branch -M main
git push -u origin main
git pull
```

■ Jangan commit rahasia

File .env, password database, token API, private key, dan credential lain tidak boleh dimasukkan ke repository publik. Gunakan .gitignore.

```
• • • .GITIGNORE
```

```
.env  
*.log  
node_modules/  
dist/
```

Dasar Web: HTTP, JSON, REST, dan API

Sebelum membuat backend, pahami bahasa komunikasi antara browser/Vue dan server Go.

14.1 Request dan response

```

CODE
CLIENT                                SERVER
|                                     |
| --- GET /api/users ----->|
|<-- 200 + JSON -----|

```

14.2 Method HTTP

Method	Makna umum	Contoh
GET	Membaca data	GET /users
POST	Membuat data	POST /users
PUT	Mengganti resource	PUT /users/10
PATCH	Mengubah sebagian	PATCH /users/10
DELETE	Menghapus	DELETE /users/10

14.3 Status code penting

Kode	Makna
200	Berhasil
201	Resource dibuat
204	Berhasil tanpa body
400	Request tidak valid
401	Belum terautentikasi
403	Tidak punya izin
404	Tidak ditemukan

Kode	Makna
409	Konflik data
422	Validasi gagal
500	Kesalahan server

14.4 JSON

```

• • • JSON
{
  "id": 1,
  "name": "Andi",
  "active": true
}
```

JSON memiliki object, array, string, number, boolean, dan null. API modern sering mengirim dan menerima JSON.

14.5 REST sebagai gaya desain

```

• • • CODE
GET    /api/tasks
GET    /api/tasks/10
POST   /api/tasks
PATCH /api/tasks/10
DELETE /api/tasks/10
```

Nama URL sebaiknya berorientasi resource. Aksi dibedakan terutama oleh HTTP method.

Backend Go: HTTP Server dan API Sederhana

Sekarang fondasi Go digunakan untuk menerima request dan menghasilkan response.

15.1 Server paling kecil

```
GO
package main

import (
    "fmt"
    "net/http"
)

func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "Halo dari Go")
    })

    fmt.Println("Server: http://localhost:8080")
    http.ListenAndServe(":8080", nil)
}
```

15.2 JSON response

```
GO
package main

import (
    "encoding/json"
    "net/http"
)

type Message struct {
    Text string `json:"text"`
}

func main() {
    http.HandleFunc("/api/hello", func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(Message{Text: "Halo"})
    })

    http.ListenAndServe(":8080", nil)
}
```

15.3 Validasi method

```
GO
if r.Method != http.MethodGet {
    http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
    return
}
```

15.4 Lapisan yang akan kamu temui

Lapisan	Tanggung jawab
Handler	HTTP request/response
Service	Aturan bisnis
Repository	Akses database
Model/DTO	Bentuk data
Middleware	Cross-cutting: auth, logging, CORS

Untuk project latihan kecil, jangan memaksakan terlalu banyak lapisan. Namun pemisahan tanggung jawab akan penting saat aplikasi membesar.

JavaScript Fundamental untuk Vue

Vue menggunakan JavaScript. Jika array, object, function, dan async/await belum dipahami, Vue akan terasa seperti magic.

16.1 Variable dan tipe

```
• • • JAVASCRIPT
const nama = "Andi";
let umur = 20;
const aktif = true;
const nilai = [80, 90, 75];
const user = { id: 1, name: "Andi" };
```

16.2 Function

```
• • • JAVASCRIPT
function tambah(a, b) {
  return a + b;
}

const kali = (a, b) => a * b;
```

16.3 Array method

```
• • • JAVASCRIPT
const angka = [1, 2, 3, 4, 5];

const ganda = angka.map(n => n * 2);
const genap = angka.filter(n => n % 2 === 0);
const total = angka.reduce((sum, n) => sum + n, 0);
```

16.4 Destructuring dan spread

```
• • • JAVASCRIPT
const user = { id: 1, name: "Andi" };
const { id, name } = user;

const updated = { ...user, name: "Budi" };
```

16.5 async/await dan fetch

```
• • • JAVASCRIPT
async function loadUsers() {
  const response = await fetch("http://localhost:8080/api/users");

  if (!response.ok) {
    throw new Error("Gagal mengambil data");
  }

  return await response.json();
}
```

Vue 3 Fundamental

Vue menghubungkan state JavaScript ke tampilan HTML secara reaktif.

17.1 Component sederhana

```

• • • VUE
<script setup>
import { ref } from "vue";

const count = ref(0);
</script>

<template>
  <button @click="count++">
    Count: {{ count }}
  </button>
</template>

```

ref membuat nilai reaktif. Ketika count berubah, bagian template yang memakai count ikut diperbarui.

17.2 Conditional rendering

```

• • • VUE
<p v-if="loading">Memuat...</p>
<p v-else-if="error">{{ error }}</p>
<ul v-else>
  <li v-for="user in users" :key="user.id">
    {{ user.name }}
  </li>
</ul>

```

17.3 Form binding

```

• • • VUE
<script setup>
import { ref } from "vue";
const name = ref("");
</script>

<template>
  <input v-model="name" />
  <p>{{ name }}</p>
</template>

```

17.4 Lifecycle + fetch

```
• • • VUE
<script setup>
import { onMounted, ref } from "vue";

const users = ref([]);
const loading = ref(true);

onMounted(async () => {
  try {
    const response = await fetch("http://localhost:8080/api/users");
    users.value = await response.json();
  } finally {
    loading.value = false;
  }
});
</script>
```

17.5 Konsep berikutnya

- props dan emit
- computed
- watch
- Vue Router
- composable
- state management (mis. Pinia jika project membutuhkan)
- form validation
- error/loading/empty state

SQL dan PostgreSQL

Database menyimpan state yang harus bertahan setelah aplikasi ditutup.

18.1 Model relasional

Data disimpan dalam table. Setiap row adalah record, setiap column memiliki tipe. Primary key mengidentifikasi row. Foreign key menghubungkan table.

18.2 Membuat table

```
• • • SQL
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

18.3 CRUD SQL

```
• • • SQL
INSERT INTO users (name, email)
VALUES ('Andi', 'andi@example.com');

SELECT id, name, email
FROM users
ORDER BY id DESC;

UPDATE users
SET name = 'Andi Baru'
WHERE id = 1;

DELETE FROM users
WHERE id = 1;
```

18.4 WHERE dan parameter

Di backend, jangan membangun SQL dengan menggabungkan input pengguna ke string. Gunakan parameter query yang disediakan driver database untuk menghindari SQL injection.

```
• • • SQL
SELECT id, name, email
FROM users
WHERE email = $1;
```

18.5 Relasi

```
• • • SQL
CREATE TABLE tasks (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL REFERENCES users(id),
  title TEXT NOT NULL,
  done BOOLEAN NOT NULL DEFAULT FALSE
);
```

18.6 JOIN

```
• • • SQL
SELECT
  tasks.id,
  tasks.title,
  users.name AS owner_name
FROM tasks
JOIN users ON users.id = tasks.user_id;
```

18.7 Transaction

```
• • • SQL
BEGIN;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT;
```

Transaction menjaga beberapa operasi agar diperlakukan sebagai satu unit: semuanya berhasil atau dibatalkan.

DBeaver, Postman, dan Browser DevTools

Tools ini membantu melihat apa yang sedang terjadi tanpa menebak.

19.1 DBeaver

- Buat connection ke PostgreSQL.
- Gunakan SQL Editor untuk mengetik query sendiri.
- Lihat struktur table, index, constraint, dan data.
- Jangan bergantung pada tombol GUI; kemampuan SQL tetap utama.

19.2 Postman / Insomnia

Gunakan untuk menguji backend sebelum frontend selesai.

```
CODE
GET http://localhost:8080/api/tasks

POST http://localhost:8080/api/tasks
Content-Type: application/json

{
  "title": "Belajar Go"
}
```

19.3 Browser DevTools

Panel	Kegunaan
Elements	Melihat DOM dan CSS
Console	Log JavaScript dan error
Network	HTTP request/response, status, payload
Application	Storage, cache, service worker
Responsive mode	Menguji layout berbagai ukuran layar

■ Saat API gagal

Buka Network. Periksa URL, method, status code, request payload, response body, dan header. Jangan langsung menyimpulkan "Vue error" atau "Go error".

Docker Fundamental

Docker membantu menjalankan service dengan lingkungan yang konsisten. Untuk tahap pertama, fokus pada container, image, port, volume, dan environment variable.

20.1 Konsep

Istilah	Arti
Image	Template read-only untuk membuat container
Container	Instance yang sedang/sempat berjalan
Port mapping	Menghubungkan port host ke port container
Volume	Penyimpanan yang bertahan di luar lifecycle container
Environment variable	Konfigurasi yang diberikan saat runtime

20.2 PostgreSQL via Docker Compose

```

• • •  YAML
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: belajar
      POSTGRES_USER: app
      POSTGRES_PASSWORD: ganti-password-ini
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:

```

```

• • •  TERMINAL
docker compose up -d
docker compose ps
docker compose logs db
docker compose down

```

■ Keamanan latihan

Credential contoh hanya untuk environment lokal. Untuk repository nyata, pindahkan konfigurasi sensitif ke environment/.env yang tidak dicommit.

20.3 Kapan Docker dipelajari?

Setelah kamu nyaman menjalankan Go, mengerti port, dan bisa menggunakan PostgreSQL. Docker seharusnya menyederhanakan environment, bukan menutupi konsep dasar yang belum dipahami.

Menggabungkan Vue + Go + PostgreSQL

Sekarang setiap alat memiliki tempat yang jelas dalam satu alur data.

21.1 Alur create task

```

• • • CODE
1. User mengetik judul di Vue
2. Vue POST /api/tasks dengan JSON
3. Go decode JSON
4. Go validasi title
5. Go INSERT ke PostgreSQL
6. PostgreSQL mengembalikan row
7. Go kirim 201 + JSON
8. Vue memperbarui daftar task

```

21.2 Bentuk JSON

```

• • • JSON
{
  "title": "Belajar slice Go"
}

```

21.3 DTO di Go

```

• • • GO
type CreateTaskRequest struct {
    Title string `json:"title"`
}

type Task struct {
    ID    int64 `json:"id"`
    Title string `json:"title"`
    Done  bool  `json:"done"`
}

```

21.4 Validasi

```

• • • GO
title := strings.TrimSpace(input.Title)
if title == "" {
    http.Error(w, "title wajib diisi", http.StatusBadRequest)
    return
}

```

21.5 Query parameterized

```
row := db.QueryRowContext(  
    r.Context(),  
    `INSERT INTO tasks (title)  
    VALUES ($1)  
    RETURNING id, title, done`,  
    title,  
)
```

Alur ini menggabungkan fundamental yang telah dipelajari: variable, struct, function, condition, error, HTTP, JSON, SQL, dan state frontend.

Mini Project Full-Stack: Task Tracker

Project ini cukup kecil untuk diselesaikan tetapi menyentuh seluruh stack.

22.1 Fitur minimum

- Menampilkan daftar task.
- Menambah task.
- Mengubah status selesai/belum selesai.
- Mengedit judul.
- Menghapus task.
- Loading, error, dan empty state.

22.2 Endpoint

Method	Endpoint	Fungsi
GET	/api/tasks	Daftar task
POST	/api/tasks	Tambah task
PATCH	/api/tasks/:id	Ubah task
DELETE	/api/tasks/:id	Hapus task

22.3 Struktur project yang mudah dipahami

```
CODE
fullstack-task/
├── backend/
│   ├── cmd/api/main.go
│   ├── internal/
│   │   ├── handler/
│   │   ├── service/
│   │   └── repository/
│   ├── go.mod
│   └── go.sum
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── views/
│   │   └── App.vue
│   ├── package.json
│   └── compose.yaml
└── docker-compose.yaml
```

22.4 Urutan implementasi

1. Buat schema PostgreSQL.

2. Buat backend GET /api/tasks.
3. Uji GET melalui Postman.
4. Buat POST + validasi.
5. Tambahkan PATCH dan DELETE.
6. Buat Vue UI statis.
7. Hubungkan GET dari Vue.
8. Hubungkan form POST.
9. Hubungkan update/delete.
10. Tambahkan loading/error/empty state.
11. Tambahkan testing backend.
12. Commit per milestone.

22.5 Definition of done

- Semua endpoint mengembalikan status HTTP yang masuk akal.
- SQL menggunakan parameter.
- Backend tidak panic untuk input invalid yang normal.
- Frontend menampilkan error secara jelas.
- Data tetap ada setelah browser direfresh.
- Project dapat dijalankan ulang dengan instruksi README yang singkat.

Jadwal Belajar dan Checkpoint

Kecepatan boleh disesuaikan. Yang penting adalah lulus checkpoint, bukan sekadar menyelesaikan hari.

Minggu	Fokus	Checkpoint
1	Algoritma, variable, operator, if	Bisa ganjil/genap dan klasifikasi nilai.
2	for, array, slice, function	Bisa total, min, max, search.
3	Prima, average, testing, struct/error	Bisa tiga latihan utama tanpa melihat solusi.
4	Git, HTTP, JSON, Go HTTP	Bisa GET/POST API in-memory.
5	JavaScript fundamental + Vue	Bisa form + list + fetch API.
6	SQL + PostgreSQL + DBeaver	Bisa CRUD SQL dan foreign key.
7	Go + PostgreSQL + Postman	API CRUD tersimpan di database.
8	Docker + integrasi full-stack	Task Tracker berjalan end-to-end.

23.1 Sesi belajar 90 menit

Durasi	Aktivitas
15 mnt	Review konsep kemarin
20 mnt	Pelajari satu konsep baru
35 mnt	Ketik dan modifikasi contoh
15 mnt	Kerjakan latihan tanpa melihat contoh
5 mnt	Commit + catat hal yang belum jelas

23.2 Indikator kamu benar-benar paham

- Bisa menjelaskan konsep dengan kata sendiri.
- Bisa memprediksi output kode sederhana sebelum menjalankan.

- Bisa menulis variasi contoh tanpa menyalin.
- Bisa menemukan edge case.
- Bisa memperbaiki error dasar dari pesan compiler/runtime.

Bank Latihan + Kunci Pembahasan

Kerjakan soal terlebih dahulu. Kunci di bagian akhir bab berisi pendekatan, bukan sekadar jawaban akhir.

24.1 Level 1 - Variable dan operator

1. Buat program menghitung luas persegi panjang dari panjang dan lebar.
2. Konversi suhu Celsius ke Fahrenheit.
3. Hitung total harga 3 barang dan diskon 10%.
4. Hitung sisa pembagian 137 dengan 12.
5. Tukar isi dua variable a dan b.

24.2 Level 2 - Percabangan

1. Cek angka ganjil/genap.
2. Cek angka positif/negatif/nol.
3. Buat kategori nilai A/B/C/D/E.
4. Cek angka habis dibagi 3 dan 5 sekaligus.
5. Cek tahun kabisat dengan aturan Gregorian.

24.3 Level 3 - Loop

1. Cetak 1 sampai 100.
2. Cetak hanya kelipatan 3 dari 1 sampai 100.
3. Hitung jumlah 1 sampai n.
4. Hitung faktorial n untuk $n \geq 0$.
5. Cetak tabel perkalian sebuah angka dari x1 sampai x10.

24.4 Level 4 - Slice

1. Jumlahkan semua elemen slice.
2. Hitung banyak angka genap dan ganjil.
3. Cari nilai minimum dan maksimum.
4. Cari index pertama dari target.
5. Buat slice baru tanpa nilai negatif.
6. Hitung rata-rata nilai > 0 saja.

24.5 Level 5 - Function dan algoritma

1. Buat function isPrime.
2. Buat function average yang aman untuk slice kosong.
3. Buat function max dan min.
4. Buat function contains(numbers []int, target int) bool.
5. Buat function countPrimes(numbers []int) int.
6. Buat function secondLargest tanpa sorting.

24.6 Level 6 - Web/backend

1. Buat endpoint GET /health yang mengembalikan JSON status ok.
2. Buat GET /api/numbers yang mengembalikan slice angka sebagai JSON.
3. Buat POST /api/average yang menerima array angka dan mengembalikan rata-rata.
4. Buat POST /api/prime yang menerima satu angka dan mengembalikan apakah prima.
5. Simpan task ke PostgreSQL dan implementasikan GET/POST.

24.7 Kunci pendekatan

Faktorial

```
func factorial(n int) (int, bool) {
    if n < 0 {
        return 0, false
    }

    result := 1
    for i := 2; i <= n; i++ {
        result *= i
    }
    return result, true
}
```

Contains

```
func contains(numbers []int, target int) bool {
    for _, number := range numbers {
        if number == target {
            return true
        }
    }
    return false
}
```

Count primes

```
• • • GO
func countPrimes(numbers []int) int {
    count := 0
    for _, number := range numbers {
        if isPrime(number) {
            count++
        }
    }
    return count
}
```

Second largest - ide

Simpan dua kandidat: largest dan second. Ketika menemukan angka yang lebih besar dari largest, geser largest lama ke second. Ketika angka berada di bawah largest tetapi di atas second, perbarui second. Tentukan terlebih dahulu bagaimana menangani duplikat dan slice dengan kurang dari dua nilai berbeda.

■ Jangan buru-buru lihat kunci

Jika mentok, kembali ke input-output-pseudocode-trace table. Mencari solusi setelah 2 menit membuat otak tidak sempat membangun kemampuan problem solving.

Cheat Sheet Go

Referensi cepat setelah konsepnya sudah dipahami.

Kebutuhan	Bentuk
Variable	<code>x := 10</code>
Konstanta	<code>const Pi = 3.14</code>
If	<code>if kondisi { ... } else { ... }</code>
Loop	<code>for i := 0; i < n; i++ { ... }</code>
Range	<code>for _, v := range values { ... }</code>
Slice	<code>xs := []int{1, 2, 3}</code>
Append	<code>xs = append(xs, 4)</code>
Panjang	<code>len(xs)</code>
Map	<code>m := map[string]int{}</code>
Function	<code>func f(a int) int { return a }</code>
Struct	<code>type User struct { Name string }</code>
Pointer	<code>&x</code> dan <code>*p</code>
Error check	<code>if err != nil { ... }</code>

Template program console

```

GO
package main

import "fmt"

func main() {
    // input
    // process
    // output
    fmt.Println("selesai")
}
```

Template function aman untuk slice

```
• • • GO
func calculate(values []int) (int, bool) {
    if len(values) == 0 {
        return 0, false
    }

    // process values

    return 0, true
}
```

Cheat Sheet Git, SQL, HTTP, dan Docker

Gunakan sebagai pengingat setelah mempelajari bab terkait.

Git

```

• • • TERMINAL
git status
git add .
git commit -m "pesan"
git log --oneline
git switch -c nama-branch
git pull
git push

```

SQL

```

• • • SQL
SELECT ... FROM ... WHERE ...;
INSERT INTO ... (...) VALUES (...);
UPDATE ... SET ... WHERE ...;
DELETE FROM ... WHERE ...;

```

HTTP

Tujuan	Method	Status sukses umum
Baca	GET	200
Buat	POST	201
Ubah	PATCH/PUT	200/204
Hapus	DELETE	204

Docker

```

• • • TERMINAL
docker compose up -d
docker compose ps
docker compose logs -f
docker compose down

```

Checklist Kelulusan Fundamental

Jangan lanjut karena "sudah membaca". Lanjut ketika sebagian besar pernyataan berikut benar tanpa bantuan besar.

- ☐ Saya bisa menjelaskan variable, tipe data, operator, if, dan for dengan kata sendiri.
- ☐ Saya bisa menulis loop dan mengetahui nilai counter pada setiap iterasi.
- ☐ Saya paham index slice dimulai dari 0 dan dapat menggunakan len/range.
- ☐ Saya bisa menggunakan accumulator untuk total.
- ☐ Saya bisa menulis function dengan parameter dan return value.
- ☐ Saya bisa menangani slice kosong sebelum mengakses elemen pertama.
- ☐ Saya bisa menjelaskan mengapa $n\%i == 0$ relevan untuk bilangan prima.
- ☐ Saya bisa menghitung rata-rata tanpa kehilangan desimal.
- ☐ Saya bisa mencari nilai maksimum untuk data positif maupun negatif.
- ☐ Saya bisa menjalankan gofmt dan go test.
- ☐ Saya bisa membuat commit Git dan membaca git diff.
- ☐ Saya memahami request, response, method HTTP, status code, dan JSON.
- ☐ Saya bisa membuat server Go sederhana dan menguji endpoint dengan Postman.
- ☐ Saya mengerti JavaScript array/object/function/async-await dasar.
- ☐ Saya bisa membuat komponen Vue dengan state, form, list, dan fetch.
- ☐ Saya bisa melakukan SELECT/INSERT/UPDATE/DELETE di PostgreSQL.
- ☐ Saya mengerti container, image, port, volume, dan environment variable.
- ☐ Saya memahami alur Vue -> HTTP -> Go -> SQL -> PostgreSQL -> JSON -> Vue.

■ Target akhir tahap dasar

Jika tiga latihan inti dapat kamu selesaikan dari pseudocode sampai test tanpa menyalin solusi, fundamental awalmu sudah cukup kuat untuk masuk backend dan integrasi full-stack.

SELESAI - gunakan buku ini sebagai roadmap, bukan sebagai bahan hafalan.